生成 AI の「目」と判断 AI の「頭」で、SO-101 にキューブを運ばせる

Physical AI 応用1講座 最終課題 説明資料 ／ シミュレーター：MuJoCo ／ ロボット：SO-101（6 軸アーム）

1. 課題の概要

MuJoCo 上の SO-101 に、2 つの作業をさせました。

- **持ち上げる**：3 cm の赤いキューブを 5 cm 以上持ち上げる
- **運んで置く**：キューブを緑のゴールまで運び、中心から 10 mm 以内に置いて離れる

動きは「キューブの上へ移動」「下ろす」「つかむ」「持ち上げる」「持ち直す」「運ぶ」など、意味のある 10 個のスキルに分けています。AI は、毎回「次にどのスキルを実行するか」だけを決めます。役割は 3 つに分けました。

役割	担当	すること
目	Claude（Claude Code のセッション）	各ステップの 2 視点の画像を見て、「キューブが指にはさまれているか」など 10 項目を yes / no / unclear で答える
頭	判断 AI（System One 型）	観測とロボットの状態から、次のスキルを確率付きで選ぶ。安全確認（例：「空中で持てているか」）の yes 確率も返す
手	プログラム（mink による IK + MuJoCo）	選ばれたスキルを毎回同じように実行する。決められた順番以外の動きは拒否する

2. 目的

これまで「頭」には、有料のホスティング型判断 AI である **Jev**（TypeSafe）を使い、2 つの作業とも成功していました。今回の目的は、2026 年 10 月に無料公開された判断 AI で、Jev を置き換えられるかを確かめることです。

- **strands-decider**（AWS の研究チーム）：2B 版（Qwen3.5 ベース）、E4B 版（Gemma 4 ベース）
- **H2O-Lightning-4B**（H2O.ai）：公開ベンチマークで Jev と同程度の総合点

どれも自分の PC（Apple Silicon）で動くので、使用料がかからず、データも外に出ません。現実の現場では、通信が不安定な環境や、データを外に出せない工場などでの利用につながります。

3. 検証の方法

1. **記録した質問の再送**：Jev で成功したときに送った 19 回分の質問を、そのまま各モデルに送りました。比べたのは、Jev と同じスキルを選ぶか、合格ラインを越えるかの 2 点です。
2. **失敗を伝えたときの安全性**：「つかみ損ねた」「ゴールの上にない」など、7 か所で観測を失敗の内容に書き換えました。そのうえで、危ないスキルに進まないかを確かめました。
3. **シミュレーションでの実行**：目は Claude、頭は判断 AI として、最初から最後まで動かしました。

合格ラインは、Jev のときと同じにしました。

- スキル選択の確信度が 0.45 以上
- 安全確認の yes 確率が 0.60 以上

このどちらかを下回るか、順番に合わないスキルが選ばれたら、プログラムはその場で止まります。

4. 結果

4.1 記録した質問での比較

比べた点	Jev	strands-decider 2B	strands-decider E4B	H2O-Lightning-4B
合格ラインを越えた手（成功ランの 15 手中）	15	4	4	14
Jev と同じスキルを選んだ手（19 手中）	—	15	14	18
スキル選択の確信度（中央値）	≈1.0	0.41	0.23	0.94
失敗を伝えたときに止まれた場所（7 か所中）	7	7	7	7
1 回の応答時間（中央値）	約 0.2 秒	約 0.5 秒	約 1.5 秒	約 0.9 秒

strands-decider は、Jev と同じスキルを選ぶことは多かったのですが、確信度が低く、持ち上げる前に止まりました。H2O-Lightning-4B は、15 手中 14 手で合格しました。どのモデルも、失敗を伝えた 7 か所すべてで危ない動作を止められました。

4.2 H2O-Lightning-4B が手順を飛ばした一手と、その原因

H2O-Lightning-4B が Jev と違う選び方をしたのは、運んで置く作業の 5 手目、持ち上げた直後だけでした。Jev は「持ち直す」を選びましたが、H2O-Lightning-4B は「運ぶ」を選びました。シミュレーションで実行したときも、プログラムが順番違いとして拒否し、ここで止まりました。

原因は、AI に渡していた行動説明の文でした。

元の説明

- 持ち直す：「持ち上げた直後で、空中に持っている。運ぶ前に握り直す」
- 運ぶ：「しっかり空中に持っていて、ゴールの上にはない。ゴールの上へ運ぶ」

持ち上げた直後の様子は、この両方に当てはまります。「しっかり」持てているかどうかは、画像からは分かりません。

直した説明

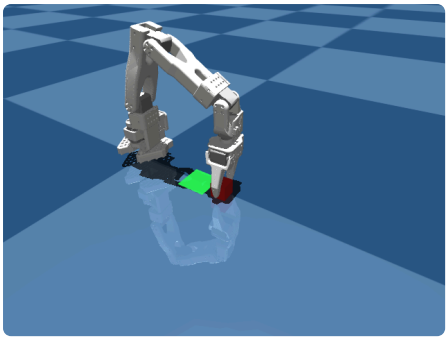
1. 順番の説明に、「1 つずつ順番どおりに進む。目的が済んでいるように見えても、スキルを飛ばさない」と書き足しました。
2. 「持ち直す」と「運ぶ」を、見た目ではなく、**直前に完了したスキル**で区別する文にしました。

持ち上げた直後に AI がつけた確率	直す前 Jev	直す前 H2O	直した後 Jev	直した後 H2O
「持ち直す」（正しい次の手）	0.79〜0.87	**0.026**	**1.00**	**0.864**
「運ぶ」	0.13〜0.21	**0.949**	0.00	0.111
記録した質問で合格した手（15 手中）	15	14	15	**15**

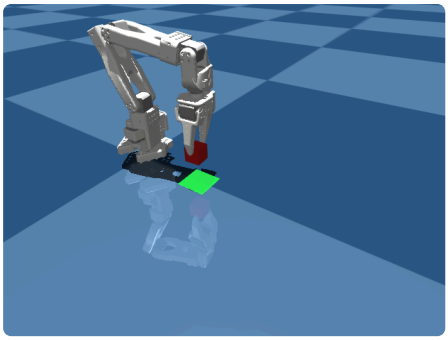
直した後は、H2O-Lightning-4B も「持ち直す」を選ぶようになりました。Jev の迷い（0.79〜0.87）もなくなりました。19 回分の質問をすべて聞き直しても、ほかの手が悪くなることはありませんでした。

4.3 シミュレーションでの実行（目：Claude、頭：H2O-Lightning-4B）

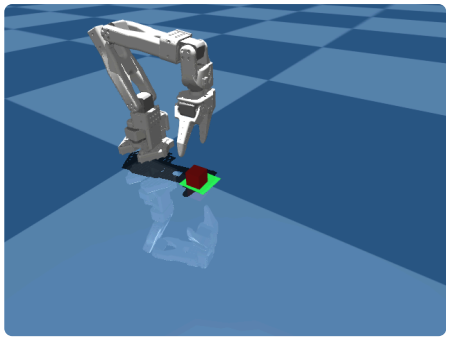
作業	Jev（比較の基準）	H2O-Lightning-4B
持ち上げる（5 cm 以上で合格）	成功 5.80 cm	**成功 5.80 cm**
運んで置く・説明を直す前	成功 7.87 mm	**5 手目で停止（順番違いを拒否）**
運んで置く・説明を直した後（10 mm 以内で合格）	—	**成功 7.87 mm（10 手すべて H2O が判断）**
1 回の判断時間	約 0.2 秒	0.8〜1.4 秒
使用料	作業 1 回あたり 1 円未満	かからない



つかむ（3 手目）



運ぶ（6 手目）



置いて離れる（9 手目）

説明を直した後の H2O-Lightning-4B による「運んで置く」（斜めカメラ。目の担当の Claude が見た画像と同じもの）

5. 工夫した点

- **役割を分けて、AI に関節角を出させない構成**：AI はスキルを選ぶだけにしました。細かい動きと安全ルール（確信度、安全確認、順番チェック）はプログラム側に固定しています。判断 AI を入れ替えても、危ない動きにならない作りです。
- **公平な比較の工夫**：途中でコードの質問文を変えていたため、Jev に実際に送った状態をそのまま記録から送り直しました。また、応答に入っているモデル名を毎回確かめました。最初は、環境変数の設定ミスで、Jev の列もローカルモデルが答えていました。モデル名を確かめることで、これに気づけました。
- **失敗のケースもわざと作って確かめる**：成功したときの判断だけでなく、失敗の観測を 7 か所で作りました。そこで止まれるかを、全モデルで確かめています。
- **サーバー設定の落とし穴を見つける**：H2O-Lightning-4B の配布サーバーは、出荷時の設定のままだと、迷っている yes / no の確率を 0.801 に書き換えて返します。これでは 0.60 の安全確認が効かなくなるので、書き換えなし（`SHIM_NOUL_FLOOR=0`）で動かしました。
- **確率の内訳から原因を探す**：違った一手で、10 個のスキルそれぞれの確率を比べました。その結果、モデルの性能不足ではなく、説明文のあいまいさが主な原因だと分かりました。説明文は、見た目ではなく「直前のスキル」で区別できる形に直しました。
- **実行の再現性**：MuJoCo と IK は決定的です。毎ステップ、それまでのスキルを最初から再実行し、指先の軌跡が記録と 1e-6 m 以内で一致することを確かめてから進めています。そのため、記録から動画を作り直せます。

6. 考察と反省

- 無料で自分の PC で動く判断 AI でも、説明文を整えれば、この作業では Jev と同じ結果を出せました。一方で、説明文が少しあいまいなだけで、正しい手の確率が 0.026 まで下がりました。**判断 AI に渡す選択肢の説明は、画像からは確かめられない言葉（「しっかり」など）に頼らない**ことが大事だと分かりました。
- strands-decider は、はい / いいえの安全確認では成功と失敗をおおむね分けられました（特に E4B 版）。しかし、5〜10 個の選択肢から選ぶ問題では確信度が低くなりました。「次は〇〇でよいか」とスキルごとに聞く形にすれば、使える可能性があります。

- どの作業も 1 回ずつの成功で、キューブとゴールの位置も 1 通りだけです。成功率や、位置を変えたときの強さは、まだ評価していません。
- 目の担当は、作業していた Claude Code のセッションが、画像を読んで観測を書く形でした。まだ自動化はしていません。次は、画像を読むモデルを API で呼んで、ループ全体を自動で回し、繰り返し評価したいと考えています。
- H2O-Lightning-4B の 1 回の判断は 0.8〜1.4 秒で、Jev（約 0.2 秒）より遅いです。今回のようにスキルを 1 つずつ選ぶ作業では問題になりませんでしたが、速い制御には向きません。

7. AI の利用と再現方法

- 推論に使った AI：Claude（画像の観測）、Jev / strands-decider 2B・E4B / H2O-Lightning-4B（スキルの判断）。学習は行っていません。
- コード、各ステップの画像、観測、確率、比較結果は、GitHub リポジトリにまとめています。手順は README を見てください。
- 動画は、H2O-Lightning-4B で「運んで置く」作業を行ったときの記録から作り直したものです。

使用した外部のもの：strands-labs/robots（SO-101 モデルの読み込み）、strands-labs/strands-decider、h2oai/h2o-lightning-4b と internet-projects/h2o-lightning-mlx（Mac 用サーバー）、MuJoCo、mink。いずれも Apache-2.0 です。Jev は TypeSafe の商用 API です。